NUCLEUS SOFTWARE · ANAND

● V3 — Architecture Locked

Forge V3

Master Document · Supersedes V3 Ideas Board + Architecture Planning Session · Last updated 2026-04-19

NORTH STAR

As close to final draft as possible.

Not replacing tester judgment. Not a generic Gherkin writer. Not a prompt wrapper.
Forge handles the mechanical thinking so the tester's domain knowledge goes where it actually matters.

WHAT FORGE IS

PLATFORM ARCHITECTURE

Forge is the platform. **CAS** is the first module.

Forge core is module-agnostic — pipeline, agentic loop, UI shell, document processing, retrieval architecture.

CAS module = CAS-specific domain knowledge, ATDD repo, stage/screen/LOB configs.

LMS, Collections, other modules plug in later. Same core, different knowledge.

HARD CONSTRAINTS

Fully offline | Open source only | No cloud calls | No licensing cost

Current: Gemma 4 E4B · Dell Latitude 5420 · 32GB RAM · No GPU

Target: Gemma 4 31B · RTX 4090 24GB · 128GB RAM · 2TB NVMe

Most agentic V3 features require Gemma 4 31B. Not viable on current hardware.

VERSION HISTORY

VERSION	STATUS	WHAT IT WAS	QUALITY
V1 Demo	FROZEN	Scenario-level stitching. Shown to VP. VP escalated to CEO/MD level. Outputs: CAS_254473.feature, CAS_256008.feature	3–5%
V2 Code	WRITTEN, UNTESTED	Step-level stitching. Role-aware retrieval (4 buckets). HyDE. Better pipeline. Never run on calibration stories.	25–40% projected

VERSION	STATUS	WHAT IT WAS	QUALITY
V3	THIS DOCUMENT	Agentic loop. Coverage thinking. Knowledge layer. Module architecture. Gemma 4 31B target.	50–80% target

THE CORE SHIFT — WHY V3 IS DIFFERENT

WHAT V1 AND V2 WERE

Translators. JIRA → Gherkin. They asked: *"What is this story supposed to do, and how do I express that in repo-authentic steps?"*

The pipeline executed steps in order. No judgment between steps. No sense of coverage. No awareness of what was missing.

WHAT V3 IS

A coverage thinker. It asks: *"What does complete coverage of this story actually look like?"*

Presence checks. Happy path. Negative paths. Validations. Conditional scenarios. Boundary values. Meaningful example rows. How many application punches are needed and why.

Forge writes scenarios the way a good tester thinks — not the way a workflow automator executes.

THE REALIZATION THAT DROVE THIS

A tester with 8 years of CAS experience saw ChatGPT write scenarios for a Waiver Allocation Master story. GPT didn't write one flow. It wrote: validate presence of menu option · validate presence of create button · validate grid shows existing records · validate fields visible on create screen.

The difference: 8 years of automation had produced journeys. GPT produced checkpoints. Presence, structure, happy path, validations — all separate, all deliberate. A single story tested properly can have 100 scenarios. Forge must think this way.

THE AGENTIC LOOP — FIVE STEPS

1 Understand — TESTER CONFIRMS

Forge reads JIRA. Makes its best attempt at identifying stage(s), LOB(s), screen(s), entities. Presents a draft — not a form to fill. Tester glances and corrects if wrong. One quick pass. This confirmation anchors everything downstream.

> **Key constraint:** A story touches multiple stages, LOBs, screens. Forge surfaces all of them — does not flatten to one each. Tester corrects, not fills.
> **Design principle:** Forge thinks, tester confirms. Any step requiring significant manual input will be skipped. The 3-amigos notes field is the tester's knowledge injection point — not optional, central.

2 Know — FORGE RUNS SILENTLY

With confirmed stage/LOB/screen context, Forge fetches relevant knowledge from the document library. Field names, conditional rules, section visibility, mandatory logic, subtype-specific behavior — for this screen, this stage, this LOB.

> **Source:** ~100 CAS documents (766-page manual + screen guides + release notes + business rule docs). The 766-page manual alone contains field-level conditional logic — e.g. Builder Details section visibility rules, subtype-specific fields, mandatory conditions per collateral subtype.
> **Mechanism:** On-demand distillation + RAPTOR. First run generates and caches. Second run retrieves directly. No manual knowledge base building required.

---

3 Scope TESTER CONFIRMS

Before writing anything, Forge identifies story scope vs ambient scope. Story scope = what this JIRA is introducing or changing. Ambient scope = everything else on the screen that exists but is not the story.

> **Story scope:** Test deeply. Full coverage layers.
> **Ambient scope:** Light touch only. Presence check. One scenario. Do not expand.
> **Why this matters:** A new field on Property Details screen should not trigger full coverage of Property Address, Builder Details, and every other section. Forge must stay inside the story. This is harder than it sounds — JIRA bleeds, dependencies exist — but the principle governs.

---

4 Write — Iteratively AGENTIC

Forge writes a first pass. Not final output — a working draft. Then reads what it wrote, identifies what's missing, and writes more. Coverage planning happens through writing, not before it.

> **Coverage layers to check per iteration:** Presence checks · Screen structure · Happy path · Validations (mandatory fields, invalid values, wrong formats) · Conditional logic (fields that appear/disappear) · Negative paths · Boundary values · Meaningful example rows.
> **Example rows:** Not placeholders. Values that actually exercise different behaviors. Subtype A gets these fields, subtype B gets different ones — examples must reflect this.

---

5 Critique AGENTIC

Forge reads its own output. Did I cover what I planned? What did I miss? What am I uncertain about? Surfaces gaps honestly. Flags them for the tester.

> **Output to tester:** Gherkin feature file + honest gap report. Not just a feature file.
> **Rule:** Do not fill gaps with noise or invented scenarios. Visible gaps are better than fabricated coverage.
> `[NEW_STEP_NOT_IN_REPO]` and `[LOW_MATCH]` markers remain. `[ROLE_GAP]` added in V3.

FULL PIPELINE — VISUAL

JIRA CSV Input EXISTING

jira_parser.py — summary, description, AC, system_process, business_scenarios, 3-amigos notes

Step 1 — Understand TESTER CONFIRMS

Forge drafts: stage(s), LOB(s), screen(s), entities from JIRA text

- → Tester reviews draft in UI — one quick glance, correct if wrong
- → Confirmation anchors all downstream steps
- → 3-amigos notes field: tester injects context not in JIRA here

Step 2 — Know V3 NEW

On-demand knowledge fetch from CAS document library

- → RAPTOR tree — hierarchical chunking of 766-page manual + screen docs
- → On-demand distillation — generates focused summary for this screen/stage/LOB, caches it
- → Outputs: field names, conditional rules, section visibility, mandatory logic per subtype

Step 3 — Scope TESTER CONFIRMS

Forge identifies story scope vs ambient scope

- → Story scope: what JIRA is introducing/changing → test deeply
- → Ambient scope: existing screen content not part of story → light touch only
- → Tester confirms or corrects. This boundary governs depth.

HyDE — Hypothesis Generation V2 CARRY

LLM generates hypothetical test scenarios from cleaned JIRA + knowledge + confirmed context

- → Extracts enriched intents: stage, screen, entities, behaviors, retrieval_query per intent
- → Intent fields: id, text, family, retrieval_query, screen_hint, entity, action_target, must_anchor_terms, must_assert_terms, forbidden_terms

Tester Reviews Intents TESTER CONFIRMS

UI shows extracted intents. Tester edits, removes, or adds. Scope boundary enforced here — intents outside story scope flagged.

Role Query Builder V2 CARRY

Per intent: 4 queries across role buckets

- → Setup · Navigation · Action · Assertion

Hybrid Retrieval EXISTING

15,000+ repo steps · FAISS 50% + PostgreSQL FTS 30% + pg_trgm 20%

- → Forbidden term filtering applied
- → Returns top 20-30 candidates per bucket with parent scenario context

Cross-Encoder Reranker V3 NEW

cross-encoder/ms-marco-MiniLM-L-6-v2 · Small · CPU-friendly · Offline

- → Re-scores top 20-30 as (intent, step) pairs — not just vector similarity
- → Catches stage/screen confusion bi-encoder misses
- → Outputs top 5-8 per bucket, better ranked

GraphRAG — Knowledge Graph Check V3 NEW

Stage → Screen → Entity → Rule graph

- → Validates: entity editable at this stage? field on this screen? action valid for role?
- → Filters or flags invalid candidates
- → New marker: [ROLE_GAP] — step invalid for this stage/entity combo

Self-RAG Gate V3 NEW

LLM critiques retrieved candidates before accepting them

→ "Is what I retrieved actually relevant to this intent?"
→ Re-retrieves with different query if candidates fail quality check
→ Skepticism built into loop — Forge can say "try again"

Step Stitcher V2 CARRY

Composes steps from 4 role buckets in Given/When/Then order

→ Markers: [NEW_STEP_NOT_IN_REPO] · [LOW_MATCH] · [ROLE_GAP]

Step 4 — Write Iteratively AGENTIC LOOP

Forge writes first pass → reads it → identifies gaps → writes more

→ Coverage layers: Presence · Structure · Happy path · Validations · Conditionals · Negatives · Boundaries
→ Example rows: meaningful values, not placeholders — exercise actual behavior differences
→ Loops until coverage plan is satisfied or gaps are acknowledged

Deterministic Assembly EXISTING

No LLM · Scenario Outline + Examples table + file header · Grounding check against 15,000+ repo steps

Step 5 — Critique AGENTIC

Forge reads its own output and assesses coverage honestly

→ Did I cover what I planned? What's missing? What am I uncertain about?
→ Produces gap report alongside feature file
→ Does not fill gaps with noise — visible gaps > fabricated coverage

Output to Tester TESTER REVIEWS

Two outputs: .feature file + gap report. Tester applies domain knowledge to gaps Forge could not resolve.

TECHNICAL COMPONENTS

EXISTING · KEEP

Hybrid Retrieval

FAISS 50% + PostgreSQL FTS 30% + pg_trgm 20%. 15,000+ repo steps. Weighted combination with parent scenario context.

Keep as-is. Foundation is solid. Reranker sits on top of it.

V3 · BUILD FIRST

Cross-Encoder Reranker

Re-scores retrieval candidates as (intent, step) pairs. Catches stage/screen confusion bi-encoder misses.

Model: cross-encoder/ms-marco-MiniLM-L-6-v2 · Small · CPU-friendly · Offline
Why first: Smallest change, highest retrieval quality impact.

V3 · KNOWLEDGE

On-Demand Distillation

Generates focused CAS knowledge summaries per screen/stage/LOB as needed. Stores and reuses. Index

V3 · KNOWLEDGE

RAPTOR Hierarchical Chunking

Recursively chunks CAS documents, summarizes groups, creates a tree. Retrieval at any level — broad stage

grows from real usage.

Feed it: 766-page manual + ~100 screen/rule docs. No manual TOML population.
Pattern: Miss → generate → cache → reuse.

concepts or specific field rules.

Why it matters: 766-page manual has both high-level stage concepts and fine-grained field rules. RAPTOR retrieves at the right level for the intent.

V3 · ARCHITECTURE

GraphRAG — Knowledge Graph

Stage → Screen → Entity → Rule graph. Relationships are first-class. Validates entity editable at stage, field on screen, action valid for role.

New marker: [ROLE_GAP]
Why: CAS is fundamentally a graph. Flat retrieval will always struggle with relationships.

V3 · AGENTIC

Self-RAG Gate

LLM critiques retrieved candidates before accepting. Re-retrieves with different query if candidates fail quality check.

Why: Currently retrieval returns candidates and Forge trusts them. Self-RAG adds skepticism — the system can say "these don't match, try again."

V3 · AGENTIC

Agentic Loop — LangGraph

Iterative write → critique → improve loop. Forge checks its own coverage and loops until satisfied or gaps acknowledged.

Framework: LangGraph or equivalent open source offline agent framework.
Requires: Gemma 4 31B for reliable agent behavior.

V2 · CARRY FORWARD

HyDE + Role Query Builder

Hypothetical scenario generation before retrieval. 4-bucket role queries: setup, navigation, action, assertion.

Status: Written in V2, never tested. Run on calibration stories before V3 starts.

BUILD ORDER

01 Run V2 on calibration stories first
CAS-256008 and CAS-254473. Don't judge quality deeply — just see if it runs, what role buckets look like, what breaks. ~1 hour. Tells you what to fix before adding V3 components.

02 Cross-encoder reranker
Smallest code change, highest retrieval quality impact. Can run on current hardware. Validates approach before adding heavier components.

03 On-demand distillation + RAPTOR
Feed CAS document library. Let knowledge layer start self-building. These two work together — RAPTOR for structure, distillation for caching.

04 GraphRAG knowledge graph
Big structural piece. Stage → Screen → Entity → Rule. Enables [ROLE_GAP] marker and deep validation of candidate steps.

05 Self-RAG + Agentic loop (requires Gemma 4 31B)
The full iterative write → critique → improve loop. Not viable on current 8B model. Starts properly when target hardware arrives.

DESIGN PRINCIPLES

Forge thinks. Tester confirms.

Forge does the heavy lifting and shows its work. Tester validates quickly. Any step requiring significant tester manual input will be skipped in practice — this has already happened with earlier UI features. Design for one glance and a correction, not form-filling.

Forge core is module-agnostic.

CAS is the first module. LMS, Collections come later. If a decision is CAS-specific, it belongs in the CAS module, not Forge core. Enforce this from the foundation — legacy burden from wrong fundamentals is carried for years.

Respect story scope.

Deep coverage on story scope. Light touch on ambient scope. Forge must not go exploring the entire screen because it now knows about it. A new field is not an invitation to retest the whole module.

Visible gaps over fabricated coverage.

Fewer correct scenarios is better than more wrong ones. If Forge doesn't know, it says so. The gap report is a first-class output. Markers ([NEW_STEP_NOT_IN_REPO], [LOW_MATCH], [ROLE_GAP]) are honest signals, not failures.

Knowledge doesn't have to be complete. It has to be enough.

CAS domain knowledge mostly lives in people's heads. Forge will never have all of it. It needs enough structured knowledge per story context to reason like a good tester. The document library + 3-amigos notes + on-demand distillation is that enough.

DEFERRED — STILL UNRESOLVED

- Screen-dominance in assembly — one screen hint taking over too early. Need to keep multiple screen candidates alive longer, commit later based on combined evidence.
- Prefer newer repo step variants over older-but-correct ones when both exist in ATDD repo.
- Deeper example/value synthesis — currently fills Examples table with best available, not most meaningful values. V3 agentic loop addresses this but needs iteration.
- Scenario-family awareness — repo operates on scenario families and execution graphs. Forge still operates on flat intent lists. This abstraction mismatch is a root cause of quality issues.
- Richer intermediate representation — needs to capture: scenario family, stage/screen anchor, ordered dependency, workbook/data contract, examples segmentation.
- Tighter placeholder naming — beyond generic repo placeholders like FacilityType when better domain signals are available.
- Knowledge acquisition agent — Gemma 4 vision could screenshot CAS screens and extract field structure systematically. Organizational constraints make this a later conversation. Not current.
- Tester feedback loop — tester corrections feeding back into knowledge layer. Planned but deferred.

QUALITY TRAJECTORY

WORKING RULES — FOR ALL FUTURE SESSIONS

→ Forge core must be module-agnostic. CAS-specific logic belongs in CAS module only.

→ Coverage thinking before step stitching. Agent must plan what to test, not just how to express it.

→ Story scope must be identified and respected. Agent must not expand into ambient scope.

→ All technology choices must be offline and open source. No exceptions.

→ Prefer generic improvements over story-specific fitting. Do not overfit to calibration samples.

→ Do not optimize for scenario count. Optimize for coverage correctness.

→ Surface uncertainty honestly. Gap reports are first-class output, not failure states.

→ Verify with tests before claiming changes are safe.

→ Do not reintroduce markdown parsing into runtime logic.

→ Keep AGENTS.md updated after meaningful architectural changes.

→ Ask Anand for missing enterprise context before assuming unavailable.

→ When running shell commands, prefer cmd style over PowerShell.